

JAVA SPEZIAL

Swing Countdown Timer

Dark Mode, runde Buttons und MP3-Sound

**WindowBuilder, Fortschrittsbalken, Start/Pause/Fortsetzen, Reset,
Minuten:Sekunden-Anzeige und Sound am Ende**

In diesem Skript:

- Aufbau einer Swing-Oberfläche mit WindowBuilder
- Dark Mode mit Farben, Schriftarten und Größen
- Runde Buttons mit eigener Klasse
- Countdown mit Minuten und Sekunden
- Start, Pause, Fortsetzen und Reset
- Fortschrittsbalken und MP3-Sound am Ende
- Projektstruktur in Eclipse mit JLayer

Inhaltsverzeichnis

1	Einleitung	4
1.1	Motivation	4
1.2	Aufbau der Arbeit	4
1.3	Lernziel	4
2	WindowBuilder in Eclipse installieren	4
2.1	Installation über den Eclipse Marketplace	4
2.2	Nach der Installation	5
2.3	Was Sie danach tun können	5
3	JLayer für MP3-Sound einbinden	5
3.1	JLayer herunterladen	5
3.2	JLayer in Eclipse einbinden	5
3.3	Import im Java-Code	6
3.4	Warum dieser Schritt wichtig ist	6
4	Projektstruktur in Eclipse	6
4.1	Vorgeschlagener Projektaufbau	6
4.2	Aufgabe der einzelnen Dateien	6
4.3	Ordner <code>src</code>	6
4.4	Ordner <code>lib</code>	7
4.5	Projektwurzel	7
4.6	Warum eine klare Struktur wichtig ist	7
5	WindowBuilder installieren und starten	7
5.1	Installation über den Eclipse Marketplace	7
5.2	Nach der Installation	7
5.3	Was Sie danach tun können	8
6	Swing-Oberfläche im WindowBuilder anlegen	8
6.1	Neue JFrame-Klasse erstellen	8
6.2	Design-Ansicht öffnen	8
6.3	Fenstereigenschaften einstellen	9
6.4	Nächster Schritt	9
7	Komponenten der Oberfläche	9
7.1	Benötigte Komponenten	9
7.2	Empfohlene Namen der Komponenten	10
7.3	Anordnung der Oberfläche	10
7.4	Nächster Schritt	10
8	Farben, Schriftarten und Größen	10
8.1	Farbpalette	11
8.2	Schriftarten	11
8.3	Größen der Komponenten	11
8.4	Nächster Schritt	11
9	Projektstruktur in Eclipse	12
9.1	Vorgeschlagener Projektaufbau	12
9.2	Aufgabe der einzelnen Dateien	12
9.3	Ordner <code>src</code>	12
9.4	Ordner <code>lib</code>	12

9.5	Projektwurzel	12
9.6	Warum eine klare Struktur wichtig ist	13
10	RoundButton-Klasse für runde Buttons	13
10.1	Aufbau der Klasse	13
10.2	Wichtige Eigenschaften	13
10.3	Beispielhafte Verwendung	13
10.4	Größe und Farbe	13
10.5	Nächster Schritt	14
11	Countdown-Logik mit Timer	14
11.1	Grundidee des Countdowns	14
11.2	Wichtige Variablen	14
11.3	Ablauf beim Starten	14
11.4	Ablauf während des Countdowns	15
11.5	Ablauf bei Null	15
11.6	Nächster Schritt	15
12	Start-, Pause-, Fortsetzen- und Reset-Funktion	15
12.1	Start-Funktion	15
12.2	Pause-Funktion	16
12.3	Fortsetzen-Funktion	16
12.4	Reset-Funktion	16
12.5	Nächster Schritt	16
13	Anzeige der Zeit im Format MM:SS	17
13.1	Berechnung von Minuten und Sekunden	17
13.2	Formatierte Ausgabe	17
13.3	Methode zur Aktualisierung	17
13.4	Nächster Schritt	17
14	Fortschrittsbalken	17
14.1	Grundidee	18
14.2	Berechnung des Prozentwertes	18
14.3	Initialisierung des Balkens	18
14.4	Aktualisierung während des Countdowns	18
14.5	Nächster Schritt	18
15	MP3-Sound am Ende des Countdowns	18
15.1	Sounddatei vorbereiten	19
15.2	Methode zum Abspielen der MP3-Datei	19
15.3	Aufruf am Ende des Countdowns	19
15.4	Warum ein eigener Thread?	19
15.5	Nächster Schritt	19
16	Fazit	20
16.1	Was das Projekt zeigt	20
16.2	Mögliche Erweiterungen	20
16.3	Abschluss	20
17	RoundButton-Klasse für runde Buttons	20
17.1	Aufbau der Klasse	21
17.2	Wichtige Eigenschaften	21
17.3	Beispielhafte Verwendung	21
17.4	Größe und Farbe	21

17.5 Nächster Schritt	21
18 Quellcode der Hauptklasse	21
18.1 Aufbau der Klasse	22
18.2 Wichtige Bestandteile	22
18.3 Kompletter Quellcode	22
18.4 Hinweis zur Verwendung	26
18.5 Nächster Schritt	26
19 RoundButton-Klasse	26
19.1 Quellcode der Klasse	26
19.2 Wichtige Eigenschaften	27
19.3 Verwendung im Hauptfenster	28
20 Testfälle	28
20.1 Testfall 1: Gültige Eingabe und normaler Countdown	28
20.2 Testfall 2: Pause und Fortsetzen	28
20.3 Testfall 3: Reset während des Countdowns	29
20.4 Testfall 4: Ungültige Eingabe	29
20.5 Testfall 5: Sekunden größer als 59	29
20.6 Testfall 6: MP3-Datei fehlt	29
20.7 Zusammenfassung	29
21 Ausblick	30
21.1 Mögliche Erweiterungen	30
21.2 Technische Erweiterungen	30
21.3 Einordnung im Projekt	30
22 Zusammenfassung	30
22.1 Was im Projekt umgesetzt wurde	31
22.2 Mögliche Erweiterungen	31
22.3 Abschluss	31
A Anhang: Quellcode und Erläuterungen	32
A.1 Inhalt des Anhangs	32
A.2 Hinweis zur Lesbarkeit	32
B Quellcode der Hauptklasse	32
B.1 Vollständiger Quellcode	32
B.2 Erklärung der wichtigsten Teile	36
B.3 Hinweis	36
C RoundButton-Klasse	36
C.1 Vollständiger Quellcode	37
C.2 Erklärung der wichtigsten Teile	37
C.3 Hinweis	38
D Wichtige Methoden im Überblick	38
D.1 Methode <code>updateDisplay()</code>	38
D.2 Methode <code>playMp3(String filePath)</code>	38
D.3 Methode <code>styleField(JTextField field)</code>	39
D.4 Methode <code>styleRoundButton(RoundButton button, Color color)</code>	39
D.5 Timer-Logik im Konstruktor	39

1 Einleitung

Grafische Benutzeroberflächen gehören zu den wichtigsten Bestandteilen moderner Desktop-Anwendungen. In diesem Projekt wird ein Countdown-Timer mit **Swing** umgesetzt, der eine übersichtliche Oberfläche, eine Zeitsteuerung mit Start, Pause, Fortsetzen und Reset sowie einen akustischen Hinweis am Ende des Countdowns bietet.

Merkkasten

Ziel des Projekts

Ziel ist es, eine funktionierende und zugleich gut strukturierte Java-Anwendung zu erstellen, die zeigt, wie sich Swing, WindowBuilder und eine externe MP3-Bibliothek sinnvoll kombinieren lassen.

1.1 Motivation

Ein Countdown-Timer ist ein gutes Beispiel für eine kleine, aber praxisnahe Desktop-Anwendung. Er verbindet mehrere wichtige Java-Themen miteinander: grafische Oberflächen, Ereignisverarbeitung, eigene Klassen, Timer-Logik und die Einbindung externer Bibliotheken.

1.2 Aufbau der Arbeit

Die Arbeit ist so gegliedert, dass zunächst die Entwicklungsumgebung und die verwendeten Werkzeuge vorgestellt werden. Danach folgen die Beschreibung der Oberfläche, die Umsetzung der Countdown-Logik, die runden Buttons, die MP3-Wiedergabe sowie die wichtigsten Methoden und Testfälle. Zum Schluss gibt es einen Ausblick auf mögliche Erweiterungen.

1.3 Lernziel

Mit dem Projekt soll gezeigt werden, wie sich eine Java-Anwendung nicht nur funktional, sondern auch optisch ansprechend umsetzen lässt. Gleichzeitig dient der Countdown als Übungsprojekt für den Umgang mit Swing und WindowBuilder.

2 WindowBuilder in Eclipse installieren

WindowBuilder ist ein grafischer GUI-Designer für **Swing** in Eclipse. Damit können Sie Fenster, Buttons, Labels und andere Komponenten per Drag-and-drop anordnen, statt alles nur im Code zu schreiben.

Merkkasten

Warum WindowBuilder?

Für Ihr Countdown-Projekt ist WindowBuilder besonders praktisch, weil Sie damit die Oberfläche schnell aufbauen und anschließend die Logik für Countdown, Pause, Reset und MP3-Sound im Java-Code ergänzen können.

2.1 Installation über den Eclipse Marketplace

1. Öffnen Sie **Eclipse**.
2. Gehen Sie oben im Menü auf **Help** → **Eclipse Marketplace**.
3. Geben Sie in das Suchfeld **WindowBuilder** ein.
4. Wählen Sie das passende Plugin aus und klicken Sie auf **Install**.
5. Achten Sie darauf, dass die Komponenten für **Swing Designer** mitinstalliert werden.

6. Folgen Sie dem Installationsassistenten bis zum Ende.
7. Starten Sie Eclipse anschließend neu.

2.2 Nach der Installation

Nach dem Neustart können Sie über **File** → **New** → **Other** neue WindowBuilder-Formulare erstellen. Für Ihr Projekt wählen Sie später eine **JFrame**-Klasse aus, weil der Countdown in einem eigenen Fenster laufen soll.

Stolperfallen

Typischer Fehler:

Manchmal wird WindowBuilder installiert, aber der **Swing Designer** fehlt. Dann sehen Sie zwar das Plugin, können aber keine Swing-Formulare bequem bearbeiten. Prüfen Sie deshalb bei der Installation, dass die Swing-Komponenten wirklich mit ausgewählt sind.

2.3 Was Sie danach tun können

Wenn WindowBuilder installiert ist, können Sie im nächsten Schritt:

- ein neues **JFrame**-Projekt anlegen,
- die Oberfläche für den Countdown gestalten,
- und anschließend die Java-Logik für den Timer ergänzen.

3 JLayer für MP3-Sound einbinden

Damit Ihr Countdown am Ende eine **MP3-Datei** abspielen kann, brauchen Sie in Java eine zusätzliche Bibliothek. Für dieses Projekt verwenden wir **JLayer**.

Merkkasten

Warum JLayer?

Die Standard-Java-Sound-API unterstützt MP3 nicht zuverlässig. Mit JLayer können Sie die Datei `alarm.mp3` im Countdown sauber abspielen.

3.1 JLayer herunterladen

1. Laden Sie die JLayer-JAR-Datei herunter.
2. Speichern Sie die Datei am besten in einem Ordner wie `lib` im Projekt.

3.2 JLayer in Eclipse einbinden

1. Klicken Sie im Projekt mit der rechten Maustaste auf den Projektnamen.
2. Wählen Sie **Build Path** → **Configure Build Path**.
3. Öffnen Sie den Reiter **Libraries**.
4. Klicken Sie auf **Add External JARs...**
5. Wählen Sie die JLayer-JAR-Datei aus.
6. Bestätigen Sie mit **Apply and Close**.

3.3 Import im Java-Code

Wenn die Bibliothek eingebunden ist, können Sie im Code die Klasse `Player` importieren:

```
1 import javazoom.jl.player.Player;
```

Stolperfallen

Typischer Fehler:

Wenn Eclipse die Klasse `Player` nicht erkennt, ist die JAR-Datei meist nicht korrekt im Build Path eingebunden. Prüfen Sie dann die Projektbibliotheken noch einmal.

3.4 Warum dieser Schritt wichtig ist

Ohne `JLayer` können Sie zwar den Countdown bauen, aber der gewünschte **MP3-Sound am Ende** würde nicht funktionieren. Deshalb sollte die Bibliothek vor dem eigentlichen Programmcode eingebunden werden.

4 Projektstruktur in Eclipse

Für das Countdown-Projekt empfiehlt sich eine klare und einfache Projektstruktur. So behalten Sie den Überblick über die Java-Klassen, die Sounddatei und die eingebundenen Bibliotheken.

Merkkasten

Empfohlene Struktur Trennen Sie die grafische Oberfläche, die Button-Klasse und die Sounddatei sauber voneinander. Das macht das Projekt übersichtlicher und erleichtert spätere Änderungen.

4.1 Vorgeschlagener Projektaufbau

Eine sinnvolle Struktur kann so aussehen:

```
MeinProjekt/  
src/  
CountdownWindowBuilder.java  
RoundButton.java  
alarm.mp3  
lib/  
jl1.0.1.jar
```

4.2 Aufgabe der einzelnen Dateien

- `CountdownWindowBuilder.java`: Hauptklasse mit Fenster, Oberfläche und Countdown-Logik.
- `RoundButton.java`: Eigene Klasse für runde Buttons.
- `alarm.mp3`: Sounddatei, die am Ende des Countdowns abgespielt wird.
- `jl1.0.1.jar`: Externe Bibliothek für die MP3-Wiedergabe.

4.3 Ordner `src`

Im Ordner `src` liegen alle Java-Quellen. Dort speichern Sie sowohl die Hauptklasse als auch die Hilfsklassen. Eclipse erkennt diesen Ordner normalerweise automatisch als Quellordner.

4.4 Ordner lib

Der Ordner `lib` ist für externe Bibliotheken gedacht. Hier können Sie die JLayer-JAR-Datei ablegen, damit das Projekt sauber organisiert bleibt.

4.5 Projektwurzel

Die Datei `alarm.mp3` kann direkt im Projektstamm liegen. Dann ist der Zugriff im Code besonders einfach, solange die Datei im richtigen Arbeitsverzeichnis gefunden wird.

Stolperfallen

Typischer Fehler: Wenn die MP3-Datei nicht gefunden wird, liegt sie oft nicht im erwarteten Arbeitsverzeichnis. Prüfen Sie deshalb genau, wo Eclipse das Programm startet und ob der Dateipfad stimmt.

4.6 Warum eine klare Struktur wichtig ist

Eine saubere Projektstruktur hilft Ihnen dabei, den Code besser zu verstehen und Fehler schneller zu finden. Gerade bei einem GUI-Projekt mit mehreren Klassen, einer Sounddatei und einer externen Bibliothek ist Ordnung besonders wichtig.

5 WindowBuilder installieren und starten

WindowBuilder ist ein grafischer Editor für Swing-Oberflächen in Eclipse. Damit können Sie Fenster, Buttons, Labels und andere Komponenten per Drag-and-drop anordnen und anschließend die Logik im Java-Code ergänzen.

Merkkasten

Warum WindowBuilder?

Für das Countdown-Projekt ist WindowBuilder besonders praktisch, weil Sie damit die Benutzeroberfläche schnell aufbauen können. Die eigentliche Countdown-Logik, der Fortschrittsbalken und die MP3-Wiedergabe ergänzen Sie danach im Code.

5.1 Installation über den Eclipse Marketplace

1. Öffnen Sie Eclipse.
2. Gehen Sie im Menü auf **Help** → **Eclipse Marketplace**.
3. Geben Sie in das Suchfeld **WindowBuilder** ein.
4. Wählen Sie das Plugin aus und klicken Sie auf **Install**.
5. Achten Sie darauf, dass die Komponenten für **Swing Designer** mitinstalliert werden.
6. Folgen Sie dem Installationsassistenten bis zum Ende.
7. Starten Sie Eclipse anschließend neu.

5.2 Nach der Installation

Nach dem Neustart können Sie über **File** → **New** → **Other** neue WindowBuilder-Formulare erstellen. Für Ihr Projekt wählen Sie später eine **JFrame**-Klasse aus, weil der Countdown in einem eigenen Fenster laufen soll.

Stolperfallen

Typischer Fehler:

Manchmal wird WindowBuilder installiert, aber der Swing Designer fehlt. Dann können Sie zwar das Plugin sehen, aber keine Swing-Formulare bequem bearbeiten. Prüfen Sie deshalb bei der Installation, dass die Swing-Komponenten wirklich mit ausgewählt sind.

5.3 Was Sie danach tun können

Wenn WindowBuilder installiert ist, können Sie im nächsten Schritt:

- ein neues **JFrame**-Projekt anlegen,
- die Oberfläche für den Countdown gestalten,
- und anschließend die Java-Logik für den Timer ergänzen.

6 Swing-Oberfläche im WindowBuilder anlegen

Nachdem WindowBuilder installiert ist, legen Sie die grafische Oberfläche für den Countdown als **JFrame** an. Diese Klasse bildet das Hauptfenster der Anwendung und enthält später alle sichtbaren Elemente wie Zeitanzeige, Eingabefelder, Fortschrittsbalken und Buttons.

Merkkasten

Warum ein JFrame?

Ein **JFrame** ist das Standardfenster für Desktop-Anwendungen in Swing. Für einen Countdown mit eigener Oberfläche ist es die passende Basis.

6.1 Neue JFrame-Klasse erstellen

1. Klicken Sie in Eclipse mit der rechten Maustaste auf Ihr Projekt.
2. Wählen Sie **New** → **Other**.
3. Öffnen Sie den Bereich **WindowBuilder** → **Swing Designer**.
4. Wählen Sie **JFrame** aus.
5. Geben Sie der Klasse einen Namen, zum Beispiel **CountdownWindowBuilder**.
6. Bestätigen Sie mit **Finish**.

6.2 Design-Ansicht öffnen

Nach dem Anlegen der Klasse öffnet sich die Datei im Editor. Dort sehen Sie in der Regel zwei Reiter:

- **Design** für die grafische Bearbeitung,
- **Source** für den Java-Code.

Im Reiter **Design** können Sie die Oberfläche visuell aufbauen. Im Reiter **Source** ergänzen Sie später die Logik für Countdown, Pause, Reset und MP3-Sound.

6.3 Fenstereigenschaften einstellen

Für das Hauptfenster sind folgende Einstellungen sinnvoll:

- **Titel:** Countdown
- **Größe:** etwa 560 x 420
- **Resizable:** false
- **Position:** zentriert auf dem Bildschirm

Stolperfallen

Typischer Fehler:

Wenn Sie die Fenstergröße später im Code und zusätzlich im Designer ändern, kann das Layout unerwartet aussehen. Legen Sie die Größe deshalb möglichst nur an einer Stelle fest.

6.4 Nächster Schritt

Sobald das JFrame angelegt ist, können Sie im nächsten Schritt die einzelnen Komponenten wie Titel, Zeitlabel, Eingabefelder und Buttons in die Oberfläche einfügen.

7 Komponenten der Oberfläche

Für den Countdown benötigen Sie nur wenige, aber klar strukturierte Swing-Komponenten. Die Oberfläche sollte übersichtlich bleiben, damit die Bedienung einfach ist und die Zeitanzeige sofort ins Auge fällt.

Merkkasten

Ziel der Oberfläche Die Oberfläche soll die aktuelle Zeit deutlich anzeigen, die Eingabe von Minuten und Sekunden ermöglichen und die Steuerung über Start, Pause, Fortsetzen und Reset einfach machen.

7.1 Benötigte Komponenten

Für das Projekt verwenden Sie die folgenden Elemente:

- **JLabel** für den Titel Countdown
- **JLabel** für die Zeitanzeige 00:00
- **JProgressBar** für den Fortschritt
- **JLabel** für Minuten:
- **JTextField** für die Minuten-Eingabe
- **JLabel** für Sekunden:
- **JTextField** für die Sekunden-Eingabe
- **Button** für Start, Pause und Fortsetzen
- **Button** für Reset

7.2 Empfohlene Namen der Komponenten

Damit Sie die Komponenten im Code leicht ansprechen können, sollten Sie ihnen eindeutige Namen geben:

- titleLabel
- timeLabel
- progressBar
- minutesField
- secondsField
- startPauseButton
- resetButton

7.3 Anordnung der Oberfläche

Die Komponenten lassen sich sinnvoll in drei Bereiche aufteilen:

- **oben:** Titel und Zeitanzeige
- **Mitte:** Fortschrittsbalken
- **unten:** Eingabefelder und Buttons

Diese Struktur sorgt für eine klare Trennung zwischen Anzeige und Bedienung.

Stolperfallen

Typischer Fehler: Wenn zu viele Komponenten ohne klare Struktur in das Fenster gesetzt werden, wirkt die Oberfläche schnell unübersichtlich. Nutzen Sie deshalb Panels und Layouts, statt alle Elemente direkt ins Hauptfenster zu legen.

7.4 Nächster Schritt

Im nächsten Abschnitt können Sie die **Farben, Schriftarten und Größen** für diese Komponenten festlegen.

8 Farben, Schriftarten und Größen

Damit der Countdown modern und gut lesbar wirkt, sollten Sie ein einheitliches dunkles Design verwenden. Wichtig ist vor allem ein klarer Kontrast zwischen Hintergrund, Text und Bedienelementen.

Merkkasten

Warum Design wichtig ist?

Ein gutes Design verbessert nicht nur die Optik, sondern auch die Bedienbarkeit. Die Zeitanzeige sollte sofort auffallen, während Eingabefelder und Buttons klar erkennbar bleiben.

8.1 Farbpalette

Für das Dark-Mode-Layout eignet sich folgende Farbwahl:

- **Hintergrund:** #18181C
- **Panels:** #202026
- **Text:** #EAEAEA
- **Akzentfarbe für die Zeit:** #467FFF
- **Start-Button:** #2E7D32
- **Reset-Button:** #C62828
- **Eingabefelder:** #2D2D34
- **Rahmen:** #50505A

8.2 Schriftarten

Für eine klare und moderne Darstellung bieten sich folgende Schriftgrößen an:

- **Titel:** SansSerif, **Bold**, 30
- **Zeit-Anzeige:** SansSerif, **Bold**, 58
- **Labels:** SansSerif, Plain, 15
- **Eingabefelder:** SansSerif, Plain, 16
- **Buttons:** SansSerif, **Bold**, 14

8.3 Größen der Komponenten

Die folgenden Größen haben sich für das Projekt bewährt:

- **JFrame:** 560 x 420
- **Zeitanzeige:** möglichst groß, etwa 58 pt
- **ProgressBar:** Höhe etwa 26 px
- **Eingabefelder:** etwa 60 x 32 px
- **Runde Buttons:** etwa 90 x 90 px

Stolperfallen

Typischer Fehler:

Wenn Schriftgrößen und Komponentenbreiten nicht zusammenpassen, kann Text abgeschnitten wirken. Prüfen Sie deshalb nach dem Einfügen der Komponenten immer die Darstellung im Designer.

8.4 Nächster Schritt

Im nächsten Abschnitt können Sie die **Projektstruktur in Eclipse** und die einzelnen Java-Klassen genauer beschreiben.

9 Projektstruktur in Eclipse

Für das Countdown-Projekt empfiehlt sich eine klare und einfache Projektstruktur. So behalten Sie den Überblick über die Java-Klassen, die Sounddatei und die eingebundenen Bibliotheken.

Merkkasten

Empfohlene Struktur

Trennen Sie die grafische Oberfläche, die Button-Klasse und die Sounddatei sauber voneinander. Das macht das Projekt übersichtlicher und erleichtert spätere Änderungen.

9.1 Vorgeschlagener Projektaufbau

Eine sinnvolle Struktur kann so aussehen:

```
MeinProjekt/  
src/  
CountdownWindowBuilder.java  
RoundButton.java  
alarm.mp3  
lib/  
j11.0.1.jar
```

9.2 Aufgabe der einzelnen Dateien

- `CountdownWindowBuilder.java`: Hauptklasse mit Fenster, Oberfläche und Countdown-Logik.
- `RoundButton.java`: Eigene Klasse für runde Buttons.
- `alarm.mp3`: Sounddatei, die am Ende des Countdowns abgespielt wird.
- `j11.0.1.jar`: Externe Bibliothek für die MP3-Wiedergabe.

9.3 Ordner `src`

Im Ordner `src` liegen alle Java-Quellen. Dort speichern Sie sowohl die Hauptklasse als auch die Hilfsklassen. Eclipse erkennt diesen Ordner normalerweise automatisch als Quellordner.

9.4 Ordner `lib`

Der Ordner `lib` ist für externe Bibliotheken gedacht. Hier können Sie die JLayer-JAR-Datei ablegen, damit das Projekt sauber organisiert bleibt.

9.5 Projektwurzel

Die Datei `alarm.mp3` kann direkt im Projektstamm liegen. Dann ist der Zugriff im Code besonders einfach, solange die Datei im richtigen Arbeitsverzeichnis gefunden wird.

Stolperfallen

Typischer Fehler:

Wenn die MP3-Datei nicht gefunden wird, liegt sie oft nicht im erwarteten Arbeitsverzeichnis. Prüfen Sie deshalb genau, wo Eclipse das Programm startet und ob der Dateipfad stimmt.

9.6 Warum eine klare Struktur wichtig ist

Eine saubere Projektstruktur hilft Ihnen dabei, den Code besser zu verstehen und Fehler schneller zu finden. Gerade bei einem GUI-Projekt mit mehreren Klassen, einer Sounddatei und einer externen Bibliothek ist Ordnung besonders wichtig.

10 RoundButton-Klasse für runde Buttons

Die Standard-Buttons von Swing sind rechteckig. Für das Countdown-Projekt sollen die Schaltflächen optisch runder und moderner wirken. Deshalb wird eine eigene Klasse `RoundButton` verwendet, die von `JButton` erbt und den Button als Kreis zeichnet.

Merkkasten

Warum eine eigene Button-Klasse?

Mit einer eigenen Klasse können Sie das Aussehen des Buttons frei gestalten. So lassen sich runde Buttons mit Hover-Effekt, angepasster Farbe und besserer Darstellung umsetzen.

10.1 Aufbau der Klasse

Die Klasse `RoundButton` erweitert `JButton`. Dadurch bleibt das Verhalten eines normalen Buttons erhalten, aber die Darstellung wird angepasst.

- Der Button wird als Kreis oder runde Fläche gezeichnet.
- Die Standard-Hintergrundfläche wird deaktiviert.
- Der Text bleibt sichtbar und zentriert.
- Bei Mausbewegung kann ein Hover-Effekt angezeigt werden.

10.2 Wichtige Eigenschaften

Für die runde Darstellung werden folgende Einstellungen verwendet:

- `setContentAreaFilled(false)`
- `setFocusPainted(false)`
- `setBorderPainted(false)`
- `setOpaque(false)`

Zusätzlich wird die Methode `paintComponent(...)` überschrieben, damit der Button mit einer runden Fläche gezeichnet werden kann.

10.3 Beispielhafte Verwendung

Im Hauptfenster können Sie dann statt eines normalen `JButton` einen `RoundButton` verwenden:

```
1 startPauseButton = new RoundButton("Start");  
2 resetButton = new RoundButton("Reset");
```

10.4 Größe und Farbe

Damit der runde Effekt gut sichtbar ist, sollte der Button ungefähr gleich breit und hoch sein. Eine Größe von etwa 90 x 90 px ist dafür gut geeignet. Die Farbe kann je nach Funktion angepasst werden, zum Beispiel grün für Start und rot für Reset.

Stolperfallen

Typischer Fehler: Wenn die Breite und Höhe des Buttons stark voneinander abweichen, wirkt der Button nicht rund, sondern eher oval oder verzerrt. Achten Sie deshalb auf ähnliche Werte für Breite und Höhe.

10.5 Nächster Schritt

Im nächsten Abschnitt können Sie die **Countdown-Logik** mit Timer, Start, Pause, Fortsetzen und Reset beschreiben.

11 Countdown-Logik mit Timer

Die eigentliche Funktion des Programms ist der Countdown. Dafür wird in Swing ein **Timer** verwendet, der in regelmäßigen Abständen eine Aktion ausführt. In diesem Projekt wird der Timer so eingestellt, dass er jede Sekunde den verbleibenden Wert um eins reduziert.

Merkkasten

Warum ein Swing-Timer?

Der `javax.swing.Timer` eignet sich besonders gut für grafische Oberflächen, weil er die Benutzeroberfläche nicht blockiert und die Anzeige direkt aktualisiert werden kann.

11.1 Grundidee des Countdowns

Der Countdown funktioniert in mehreren Schritten:

- Der Benutzer gibt Minuten und Sekunden ein.
- Die Eingabe wird in Gesamtsekunden umgerechnet.
- Der Timer startet und reduziert jede Sekunde den Wert.
- Die Anzeige wird im Format **MM:SS** aktualisiert.
- Bei 0 stoppt der Timer und der Sound wird abgespielt.

11.2 Wichtige Variablen

Für die Countdown-Logik werden einige Variablen benötigt:

- `initialSeconds`: speichert die ursprünglich eingegebene Zeit
- `remainingSeconds`: speichert die aktuell verbleibende Zeit
- `running`: merkt sich, ob der Timer gerade läuft
- `timer`: der Swing-Timer selbst

11.3 Ablauf beim Starten

Wenn der Benutzer auf **Start** klickt, wird zunächst geprüft, ob bereits eine Zeit eingegeben wurde. Falls nicht, werden die Werte aus den Eingabefeldern gelesen und in Sekunden umgerechnet. Danach startet der Timer.

```
1  int minutes = Integer.parseInt(minutesField.getText().trim());
2  int seconds = Integer.parseInt(secondsField.getText().trim());
3  initialSeconds = minutes * 60 + seconds;
4  remainingSeconds = initialSeconds;
```

11.4 Ablauf während des Countdowns

Bei jedem Timer-Schritt wird der Wert von `remainingSeconds` um eins reduziert. Anschließend wird die Anzeige aktualisiert. Solange der Wert größer als null ist, läuft der Countdown weiter.

```
1 timer = new Timer(1000, e -> {
2     remainingSeconds--;
3     updateDisplay();
4 });
```

11.5 Ablauf bei Null

Sobald die verbleibende Zeit bei null angekommen ist, wird der Timer gestoppt und der Sound abgespielt. Zusätzlich kann die Anzeige auf **Fertig!** gesetzt werden.

```
1 if (remainingSeconds <= 0) {
2     timer.stop();
3     playMp3("alarm.mp3");
4     timeLabel.setText("Fertig!");
5 }
```

Stolperfallen

Typischer Fehler:

Wenn der Timer nicht gestoppt wird, läuft er auch bei null weiter und kann negative Werte erzeugen. Deshalb sollte die Abfrage auf `remainingSeconds <= 0` immer enthalten sein.

11.6 Nächster Schritt

Im nächsten Abschnitt können Sie die **Start-, Pause-, Fortsetzen- und Reset-Funktion** beschreiben.

12 Start-, Pause-, Fortsetzen- und Reset-Funktion

Damit der Countdown flexibel bedient werden kann, benötigt das Programm mehrere Steuerfunktionen. Der Benutzer soll den Countdown starten, zwischendurch pausieren, später fortsetzen und bei Bedarf vollständig zurücksetzen können.

Merkkasten

Warum mehrere Steuerfunktionen?

Ein Countdown ist im Alltag oft nicht nur ein einmaliger Ablauf. Mit Pause und Fortsetzen kann der Benutzer den Timer unterbrechen, ohne die bisherige Zeit zu verlieren.

12.1 Start-Funktion

Beim ersten Klick auf **Start** wird die eingegebene Zeit aus den Textfeldern gelesen. Anschließend wird der Timer gestartet und die Eingabefelder werden deaktiviert, damit während des Countdowns keine Werte verändert werden.

```
1 if (!running) {
2     if (remainingSeconds <= 0) {
3         int minutes = Integer.parseInt(minutesField.getText().trim());
4         int seconds = Integer.parseInt(secondsField.getText().trim());
5         initialSeconds = minutes * 60 + seconds;
6         remainingSeconds = initialSeconds;
7     }
```

```

8     timer.start();
9     running = true;
10    startPauseButton.setText("Pause");
11 }

```

12.2 Pause-Funktion

Wenn der Timer bereits läuft und der Benutzer erneut auf den Start-Button klickt, wird der Countdown pausiert. Die aktuelle Restzeit bleibt erhalten.

```

1  else {
2      timer.stop();
3      running = false;
4      startPauseButton.setText("Fortsetzen");
5  }

```

12.3 Fortsetzen-Funktion

Nach einer Pause kann der Countdown mit demselben Button fortgesetzt werden. Der Buttontext wechselt dann von **Fortsetzen** zurück zu **Pause**.

```

1  if (!running) {
2      timer.start();
3      running = true;
4      startPauseButton.setText("Pause");
5  }

```

12.4 Reset-Funktion

Mit der Reset-Funktion wird der Countdown vollständig zurückgesetzt. Der Timer wird gestoppt, die Anzeige springt auf den Anfangswert zurück und die Eingabefelder werden wieder aktiviert.

```

1  resetButton.addActionListener(e -> {
2      timer.stop();
3      running = false;
4      remainingSeconds = initialSeconds;
5      updateDisplay();
6      minutesField.setEnabled(true);
7      secondsField.setEnabled(true);
8      startPauseButton.setText("Start");
9  });

```

Stolperfallen

Typischer Fehler:

Wenn beim Reset nicht alle Zustände zurückgesetzt werden, kann die Oberfläche widersprüchlich wirken. Achten Sie deshalb darauf, sowohl den Timer als auch den Buttontext und die Eingabefelder korrekt zu aktualisieren.

12.5 Nächster Schritt

Im nächsten Abschnitt können Sie die **Anzeige der Zeit im Format MM:SS** beschreiben.

13 Anzeige der Zeit im Format MM:SS

Die verbleibende Zeit soll nicht nur als Gesamtsekunden angezeigt werden, sondern im gewohnten Format MM:SS. Dadurch ist der Countdown leichter lesbar und wirkt für den Benutzer übersichtlicher.

Merkkasten

Warum das Format MM:SS?

Eine Anzeige wie 02:15 ist sofort verständlich. Der Benutzer erkennt auf einen Blick, wie viele Minuten und Sekunden noch verbleiben.

13.1 Berechnung von Minuten und Sekunden

Die Umrechnung erfolgt aus der verbleibenden Gesamtzeit. Dabei wird die Minutenanzahl durch Division und die Sekundenanzahl durch den Rest der Division berechnet.

```
1  int minutes = remainingSeconds / 60;
2  int seconds = remainingSeconds % 60;
```

13.2 Formatierte Ausgabe

Damit immer zwei Stellen angezeigt werden, wird `String.format` verwendet. So erscheinen Werte wie 5 als 05.

```
1  timeLabel.setText(String.format("%02d:%02d", minutes, seconds));
```

13.3 Methode zur Aktualisierung

Die Formatierung kann in einer eigenen Methode zusammengefasst werden. Diese Methode wird nach jedem Timer-Schritt aufgerufen.

```
1  private void updateDisplay() {
2      int minutes = remainingSeconds / 60;
3      int seconds = remainingSeconds % 60;
4      timeLabel.setText(String.format("%02d:%02d", minutes, seconds));
5  }
```

Stolperfallen

Typischer Fehler:

Wenn die Anzeige nicht mit führenden Nullen formatiert wird, wirken Werte wie 1:5 unübersichtlich. Verwenden Sie deshalb immer `%02d` für Minuten und Sekunden.

13.4 Nächster Schritt

Im nächsten Abschnitt können Sie den **Fortschrittsbalken** und seine Berechnung beschreiben.

14 Fortschrittsbalken

Der Fortschrittsbalken zeigt an, wie viel von der ursprünglich eingegebenen Zeit noch übrig ist. Er ergänzt die Zeitanzeige und macht den Verlauf des Countdowns visuell besser erkennbar.

Merkkasten

Warum ein Fortschrittsbalken?

Ein Fortschrittsbalken gibt dem Benutzer sofort eine visuelle Rückmeldung. Auch ohne auf die Zahlen zu achten, ist klar erkennbar, wie weit der Countdown bereits fortgeschritten ist.

14.1 Grundidee

Zu Beginn steht der Balken auf 100%. Mit jeder verstrichenen Sekunde sinkt der Wert, bis er am Ende bei 0% ankommt.

14.2 Berechnung des Prozentwertes

Der Prozentwert wird aus der verbleibenden Zeit und der ursprünglich eingegebenen Gesamtzeit berechnet.

```
1  int percent = (int) ((remainingSeconds * 100.0) / initialSeconds);
2  progressBar.setValue(percent);
3  progressBar.setString(percent + "%");
```

14.3 Initialisierung des Balkens

Beim Start des Programms oder nach einem Reset sollte der Balken wieder auf den Anfangswert gesetzt werden.

```
1  progressBar.setValue(100);
2  progressBar.setString("100%");
```

14.4 Aktualisierung während des Countdowns

Nach jeder Sekunde wird der Balken neu berechnet und aktualisiert. Dadurch verändert sich die Anzeige synchron zur Zeit.

```
1  remainingSeconds--;
2  updateDisplay();
```

Stolperfallen

Typischer Fehler:

Wenn die ursprünglich eingegebene Zeit `initialSeconds` den Wert 0 hat, kann keine Prozentberechnung durchgeführt werden. Prüfen Sie deshalb vor der Berechnung, ob die Gesamtzeit größer als null ist.

14.5 Nächster Schritt

Im nächsten Abschnitt können Sie den **MP3-Sound am Ende des Countdowns** beschreiben.

15 MP3-Sound am Ende des Countdowns

Wenn der Countdown bei 0 angekommen ist, soll eine MP3-Datei abgespielt werden. Dafür wird die Bibliothek **JLayer** verwendet. Der Sound sorgt dafür, dass der Benutzer das Ende des Countdowns nicht nur sieht, sondern auch hört.

Merkkasten

Warum ein Sound am Ende?

Ein akustisches Signal macht das Ende des Countdowns deutlicher. Besonders dann, wenn das Fenster nicht ständig im Blick ist, ist ein Sound sehr hilfreich.

15.1 Sounddatei vorbereiten

Die MP3-Datei sollte zum Beispiel den Namen `alarm.mp3` tragen und im Projektordner oder im Arbeitsverzeichnis liegen. Der Dateiname muss im Code genau so verwendet werden, wie die Datei tatsächlich heißt.

15.2 Methode zum Abspielen der MP3-Datei

Die Wiedergabe kann in einer eigenen Methode gekapselt werden. Damit bleibt der Hauptcode übersichtlich.

```
1 private void playMp3(String filePath) {
2     new Thread(() -> {
3         try (FileInputStream fis = new FileInputStream(filePath)) {
4             Player player = new Player(fis);
5             player.play();
6         } catch (Exception e) {
7             e.printStackTrace();
8         }
9     }).start();
10 }
```

15.3 Aufruf am Ende des Countdowns

Sobald die verbleibende Zeit null erreicht, wird der Timer gestoppt und die Sound-Methode aufgerufen.

```
1 if (remainingSeconds <= 0) {
2     timer.stop();
3     playMp3("alarm.mp3");
4     timeLabel.setText("Fertig!");
5 }
```

15.4 Warum ein eigener Thread?

Die MP3-Wiedergabe läuft in einem eigenen Thread, damit die grafische Oberfläche währenddessen nicht einfriert. So bleibt das Programm auch während des Sounds bedienbar.

Stolperfallen

Typischer Fehler:

Wenn die MP3-Datei nicht gefunden wird, liegt das oft an einem falschen Pfad oder daran, dass die Datei nicht im erwarteten Arbeitsverzeichnis liegt. Prüfen Sie deshalb den Speicherort der Datei genau.

15.5 Nächster Schritt

Im nächsten Abschnitt können Sie die **Projektklasse** `RoundButton` oder direkt den **vollständigen Quellcode** beschreiben.

16 Fazit

Mit WindowBuilder, Swing und JLayer lässt sich ein übersichtlicher Countdown-Timer für den Desktop umsetzen. Das Projekt zeigt, wie sich eine grafische Oberfläche mit Eingabefeldern, Fortschrittsbalken und Steuerbuttons sauber strukturieren lässt.

Merkkasten

Wichtige Erkenntnisse Für das Projekt sind vor allem eine klare Projektstruktur, eine gut lesbare Oberfläche und eine saubere Trennung von Oberfläche und Logik entscheidend.

16.1 Was das Projekt zeigt

Das Countdown-Projekt verbindet mehrere wichtige Java-Themen:

- grafische Oberflächen mit Swing
- Gestaltung mit WindowBuilder
- Ereignisverarbeitung mit Buttons und Timer
- Arbeiten mit eigenen Klassen wie `RoundButton`
- Einbindung externer Bibliotheken für MP3-Sound

16.2 Mögliche Erweiterungen

Das Projekt kann später noch erweitert werden, zum Beispiel um:

- einen eigenen Sound-Auswahldialog
- weitere Designvarianten wie Light Mode
- eine Eingabe per Slider
- eine Speicherung letzter Einstellungen

16.3 Abschluss

Der Countdown ist damit ein gutes Beispiel dafür, wie sich Java, Swing und WindowBuilder in einem praxisnahen Projekt kombinieren lassen. Die Anwendung ist einfach zu bedienen und gleichzeitig technisch gut erweiterbar.

17 RoundButton-Klasse für runde Buttons

Die Standard-Buttons von Swing sind rechteckig. Für das Countdown-Projekt sollen die Schaltflächen optisch runder und moderner wirken. Deshalb wird eine eigene Klasse `RoundButton` verwendet, die von `JButton` erbt und den Button als Kreis zeichnet.

Merkkasten

Warum eine eigene Button-Klasse?

Mit einer eigenen Klasse können Sie das Aussehen des Buttons frei gestalten. So lassen sich runde Buttons mit Hover-Effekt, angepasster Farbe und besserer Darstellung umsetzen.

17.1 Aufbau der Klasse

Die Klasse `RoundButton` erweitert `JButton`. Dadurch bleibt das Verhalten eines normalen Buttons erhalten, aber die Darstellung wird angepasst.

- Der Button wird als Kreis oder runde Fläche gezeichnet.
- Die Standard-Hintergrundfläche wird deaktiviert.
- Der Text bleibt sichtbar und zentriert.
- Bei Mausbewegung kann ein Hover-Effekt angezeigt werden.

17.2 Wichtige Eigenschaften

Für die runde Darstellung werden folgende Einstellungen verwendet:

- `setContentAreaFilled(false)`
- `setFocusPainted(false)`
- `setBorderPainted(false)`
- `setOpaque(false)`

Zusätzlich wird die Methode `paintComponent(...)` überschrieben, damit der Button mit einer runden Fläche gezeichnet werden kann.

17.3 Beispielhafte Verwendung

Im Hauptfenster können Sie dann statt eines normalen `JButton` einen `RoundButton` verwenden:

```
1 startPauseButton = new RoundButton("Start");
2 resetButton = new RoundButton("Reset");
```

17.4 Größe und Farbe

Damit der runde Effekt gut sichtbar ist, sollte der Button ungefähr gleich breit und hoch sein. Eine Größe von etwa 90 x 90 px ist dafür gut geeignet. Die Farbe kann je nach Funktion angepasst werden, zum Beispiel grün für Start und rot für Reset.

Stolperfallen

Typischer Fehler:

Wenn die Breite und Höhe des Buttons stark voneinander abweichen, wirkt der Button nicht rund, sondern eher oval oder verzerrt. Achten Sie deshalb auf ähnliche Werte für Breite und Höhe.

17.5 Nächster Schritt

Im nächsten Abschnitt können Sie die **Countdown-Logik** mit Timer, Start, Pause, Fortsetzen und Reset beschreiben.

18 Quellcode der Hauptklasse

In diesem Abschnitt wird die komplette Hauptklasse des Projekts vorgestellt. Sie enthält das Fenster, die Oberfläche, den Timer, die Steuerung per Start, Pause, Fortsetzen und Reset sowie den Aufruf der MP3-Datei am Ende des Countdowns.

Merkkasten

Warum eine zentrale Hauptklasse?

Die Hauptklasse bündelt die gesamte Logik des Countdowns an einer Stelle. Dadurch bleibt das Projekt übersichtlich und die einzelnen Funktionen lassen sich leichter nachvollziehen.

18.1 Aufbau der Klasse

Die Klasse `CountdownWindowBuilder` erbt von `JFrame`. Dadurch kann sie direkt als Fenster verwendet werden. In der Klasse werden die grafischen Komponenten angelegt und mit der Countdown-Logik verbunden.

18.2 Wichtige Bestandteile

Die Hauptklasse enthält unter anderem:

- das Hauptfenster mit Dark-Mode-Design
- die Zeitanzeige im Format MM:SS
- die Eingabefelder für Minuten und Sekunden
- den Fortschrittsbalken
- die Buttons für Start, Pause, Fortsetzen und Reset
- die Methode zum Abspielen der MP3-Datei

18.3 Kompletter Quellcode

```
1  import javax.swing.plaf.basic.BasicLookAndFeel;
2
3  import javax.swing.*;
4  import javax.swing.border.EmptyBorder;
5  import java.awt.*;
6  import java.io.FileInputStream;
7
8  public class CountdownWindowBuilder extends JFrame {
9
10     private JPanel contentPane;
11     private JLabel titleLabel;
12     private JLabel timeLabel;
13     private JTextField minutesField;
14     private JTextField secondsField;
15     private JButton startPauseButton;
16     private JButton resetButton;
17     private JProgressBar progressBar;
18
19     private Timer timer;
20     private int initialSeconds = 0;
21     private int remainingSeconds = 0;
22     private boolean running = false;
23
24     private final Color bg = new Color(24, 24, 28);
25     private final Color panelBg = new Color(32, 32, 38);
26     private final Color text = new Color(235, 235, 235);
27     private final Color accent = new Color(70, 130, 255);
28     private final Color green = new Color(46, 125, 50);
29     private final Color red = new Color(198, 40, 40);
30     private final Color fieldBg = new Color(45, 45, 52);
```

```

31
32 public static void main(String[] args) {
33     SwingUtilities.invokeLater(() -> {
34         try {
35             CountdownWindowBuilder frame = new CountdownWindowBuilder();
36             frame.setVisible(true);
37         } catch (Exception e) {
38             e.printStackTrace();
39         }
40     });
41 }
42
43 public CountdownWindowBuilder() {
44     setTitle("Countdown");
45     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46     setBounds(100, 100, 560, 420);
47     setLocationRelativeTo(null);
48     setResizable(false);
49
50     contentPane = new JPanel();
51     contentPane.setBorder(new EmptyBorder(20, 20, 20, 20));
52     contentPane.setBackground(bg);
53     setContentPane(contentPane);
54     contentPane.setLayout(new BorderLayout(15, 15));
55
56     JPanel centerPanel = new JPanel(new GridLayout(3, 1, 12, 12));
57     centerPanel.setOpaque(false);
58
59     titleLabel = new JLabel("Countdown", SwingConstants.CENTER);
60     titleLabel.setFont(new Font("SansSerif", Font.BOLD, 30));
61     titleLabel.setForeground(text);
62
63     timeLabel = new JLabel("00:00", SwingConstants.CENTER);
64     timeLabel.setFont(new Font("SansSerif", Font.BOLD, 58));
65     timeLabel.setForeground(accent);
66
67     progressBar = new JProgressBar(0, 100);
68     progressBar.setStringPainted(true);
69     progressBar.setValue(100);
70     progressBar.setString("100%");
71     progressBar.setForeground(accent);
72     progressBar.setBackground(panelBg);
73     progressBar.setBorderPainted(false);
74     progressBar.setPreferredSize(new Dimension(0, 26));
75
76     centerPanel.add(titleLabel);
77     centerPanel.add(timeLabel);
78     centerPanel.add(progressBar);
79
80     JPanel inputPanel = new JPanel(new FlowLayout(FlowLayout.CENTER,
81         12, 0));
82     inputPanel.setOpaque(false);
83
84     JLabel minLabel = new JLabel("Minuten:");
85     minLabel.setForeground(text);
86     minLabel.setFont(new Font("SansSerif", Font.PLAIN, 15));
87
88     JLabel secLabel = new JLabel("Sekunden:");
89     secLabel.setForeground(text);
90     secLabel.setFont(new Font("SansSerif", Font.PLAIN, 15));

```

```

91     minutesField = new JTextField("0", 4);
92     secondsField = new JTextField("10", 4);
93     styleField(minutesField);
94     styleField(secondsField);
95
96     inputPanel.add(minLabel);
97     inputPanel.add(minutesField);
98     inputPanel.add(secLabel);
99     inputPanel.add(secondsField);
100
101     JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER,
102         18, 0));
103     buttonPanel.setOpaque(false);
104
105     startPauseButton = new RoundButton("Start");
106     resetButton = new RoundButton("Reset");
107     styleRoundButton(startPauseButton, green);
108     styleRoundButton(resetButton, red);
109     resetButton.setEnabled(false);
110
111     buttonPanel.add(startPauseButton);
112     buttonPanel.add(resetButton);
113
114     JPanel bottomPanel = new JPanel(new GridLayout(2, 1, 10, 10));
115     bottomPanel.setOpaque(false);
116     bottomPanel.add(inputPanel);
117     bottomPanel.add(buttonPanel);
118
119     contentPane.add(centerPanel, BorderLayout.CENTER);
120     contentPane.add(bottomPanel, BorderLayout.SOUTH);
121
122     timer = new Timer(1000, e -> {
123         remainingSeconds--;
124         updateDisplay();
125
126         if (remainingSeconds <= 0) {
127             timer.stop();
128             playMp3("alarm.mp3");
129             timeLabel.setText("Fertig!");
130             progressBar.setValue(0);
131             progressBar.setString("0%");
132             running = false;
133             startPauseButton.setText("Start");
134         }
135     });
136
137     startPauseButton.addActionListener(e -> {
138         if (!running) {
139             if (remainingSeconds <= 0) {
140                 try {
141                     int minutes = Integer.parseInt(minutesField.getText().trim());
142                     int seconds = Integer.parseInt(secondsField.getText().trim());
143
144                     if (minutes < 0 || seconds < 0 || seconds > 59) {
145                         timeLabel.setText("Ungültig");
146                         return;
147                     }
148
149                     initialSeconds = minutes * 60 + seconds;

```

```

149         if (initialSeconds <= 0) {
150             timeLabel.setText("Zeit > 0");
151             return;
152         }
153
154         remainingSeconds = initialSeconds;
155     } catch (NumberFormatException ex) {
156         timeLabel.setText("Ungültig");
157         return;
158     }
159 }
160
161     updatedDisplay();
162     timer.start();
163     running = true;
164     startPauseButton.setText("Pause");
165     resetButton.setEnabled(true);
166     minutesField.setEnabled(false);
167     secondsField.setEnabled(false);
168 } else {
169     timer.stop();
170     running = false;
171     startPauseButton.setText("Fortsetzen");
172 }
173 });
174
175     resetButton.addActionListener(e -> {
176         timer.stop();
177         running = false;
178         remainingSeconds = initialSeconds;
179         updatedDisplay();
180         timeLabel.setText("00:00");
181         progressBar.setValue(100);
182         progressBar.setString("100%");
183         minutesField.setEnabled(true);
184         secondsField.setEnabled(true);
185         startPauseButton.setText("Start");
186         resetButton.setEnabled(false);
187     });
188 }
189
190     private void updateDisplay() {
191         int minutes = remainingSeconds / 60;
192         int seconds = remainingSeconds % 60;
193         timeLabel.setText(String.format("%02d:%02d", minutes, seconds));
194
195         if (initialSeconds > 0) {
196             int percent = (int) ((remainingSeconds * 100.0) / initialSeconds
197             );
198             progressBar.setValue(percent);
199             progressBar.setString(percent + "%");
200         }
201     }
202
203     private void playMp3(String filePath) {
204         new Thread(() -> {
205             try (FileInputStream fis = new FileInputStream(filePath)) {
206                 Player player = new Player(fis);
207                 player.play();
208             } catch (Exception e) {
209                 e.printStackTrace();

```

```

209     }
210     }).start();
211 }
212
213 private void styleField(JTextField field) {
214     field.setFont(new Font("SansSerif", Font.PLAIN, 16));
215     field.setHorizontalAlignment(JTextField.CENTER);
216     field.setPreferredSize(new Dimension(60, 32));
217     field.setBackground(fieldBg);
218     field.setForeground(text);
219     field.setCaretColor(text);
220     field.setBorder(BorderFactory.createLineBorder(new Color(80, 80,
221         90), 1));
222 }
223
224 private void styleRoundButton(RoundButton button, Color color) {
225     button.setBackground(color);
226     button.setForeground(Color.WHITE);
227     button.setFont(new Font("SansSerif", Font.BOLD, 14));
228     button.setPreferredSize(new Dimension(90, 90));
229 }

```

18.4 Hinweis zur Verwendung

Wenn Sie diese Klasse in Ihr Projekt einfügen, muss die Klasse `RoundButton` zusätzlich im Projekt vorhanden sein. Außerdem muss die Datei `alarm.mp3` im richtigen Verzeichnis liegen und die `JLayer`-Bibliothek eingebunden sein.

Stolperfallen

Typischer Fehler:

Wenn die Klasse `RoundButton` fehlt oder die `JLayer`-Bibliothek nicht eingebunden ist, kann das Projekt nicht kompiliert werden. Prüfen Sie deshalb vor dem Start alle Abhängigkeiten.

18.5 Nächster Schritt

Im nächsten Abschnitt können Sie die Klasse `RoundButton` selbst vollständig darstellen.

19 RoundButton-Klasse

Die Klasse `RoundButton` sorgt dafür, dass die Buttons im Countdown-Projekt rund dargestellt werden. Sie erweitert `JButton` und überschreibt die Darstellung des Buttons.

Merkkasten

Warum eine eigene Klasse?

Mit einer eigenen Button-Klasse können Sie das Aussehen der Schaltflächen gezielt anpassen. So lassen sich runde Buttons mit Hover-Effekt und moderner Optik umsetzen.

19.1 Quellcode der Klasse

```

1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.MouseAdapter;
4  import java.awt.event.MouseEvent;
5  import java.awt.geom.Ellipse2D;

```

```

6
7 public class RoundButton extends JButton {
8     private Shape shape;
9
10    public RoundButton(String text) {
11        super(text);
12        setContentAreaFilled(false);
13        setFocusPainted(false);
14        setBorderPainted(false);
15        setOpaque(false);
16        setForeground(Color.WHITE);
17        setFont(new Font("SansSerif", Font.BOLD, 14));
18
19        addMouseListener(new MouseAdapter() {
20            @Override
21            public void mouseEntered(MouseEvent e) {
22                setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
23            }
24        });
25    }
26
27    @Override
28    protected void paintComponent(Graphics g) {
29        Graphics2D g2 = (Graphics2D) g.create();
30        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
31            RenderingHints.VALUE_ANTIALIAS_ON);
32
33        if (getModel().isArmed()) {
34            g2.setColor(getBackground().darker());
35        } else if (getModel().isRollover()) {
36            g2.setColor(getBackground().brighter());
37        } else {
38            g2.setColor(getBackground());
39        }
40
41        g2.fillOval(0, 0, getWidth(), getHeight());
42        super.paintComponent(g2);
43        g2.dispose();
44    }
45
46    @Override
47    public boolean contains(int x, int y) {
48        if (shape == null || !shape.getBounds().equals(getBounds())) {
49            shape = new Ellipse2D.Float(0, 0, getWidth(), getHeight());
50        }
51        return shape.contains(x, y);
52    }
53

```

19.2 Wichtige Eigenschaften

Die Klasse verwendet folgende Techniken:

- `setContentAreaFilled(false)` deaktiviert die Standardfläche des Buttons.
- `setFocusPainted(false)` entfernt den Fokusrahmen.
- `paintComponent(...)` zeichnet den Button als runde Fläche.
- `contains(...)` sorgt dafür, dass Klicks nur innerhalb des runden Bereichs erkannt werden.

19.3 Verwendung im Hauptfenster

Im Hauptfenster können Sie die Klasse direkt für die Steuerbuttons verwenden:

```
1 startPauseButton = new RoundButton("Start");  
2 resetButton = new RoundButton("Reset");
```

Stolperfallen

Typischer Fehler:

Wenn Breite und Höhe des Buttons stark voneinander abweichen, wirkt der Button nicht mehr rund. Achten Sie deshalb auf ähnliche Werte für beide Dimensionen.

20 Testfälle

Damit die Funktion des Countdown-Projekts überprüft werden kann, wurden mehrere Testfälle definiert. Sie decken sowohl die normale Nutzung als auch typische Fehlerfälle ab.

Merkkasten

Warum Testfälle wichtig sind?

Testfälle helfen dabei, die Funktion des Programms systematisch zu prüfen. So lässt sich erkennen, ob Countdown, Pause, Reset, Fortschrittsbalken und MP3-Wiedergabe wie geplant funktionieren.

20.1 Testfall 1: Gültige Eingabe und normaler Countdown

Ausgangssituation: Der Benutzer gibt eine gültige Zeit ein, zum Beispiel 0 Minuten und 10 Sekunden.

Erwartetes Verhalten:

- Der Countdown startet mit 00:10.
- Die Zeit wird jede Sekunde reduziert.
- Der Fortschrittsbalken sinkt schrittweise.
- Am Ende wird die MP3-Datei abgespielt.
- Die Anzeige wechselt auf **Fertig!**.

20.2 Testfall 2: Pause und Fortsetzen

Ausgangssituation: Der Countdown läuft bereits.

Erwartetes Verhalten:

- Beim Klick auf den Start-Button wird der Countdown pausiert.
- Die aktuelle Restzeit bleibt erhalten.
- Beim erneuten Klick wird der Countdown fortgesetzt.
- Die Anzeige läuft an der vorherigen Stelle weiter.

20.3 Testfall 3: Reset während des Countdowns

Ausgangssituation: Der Countdown läuft oder ist pausiert.

Erwartetes Verhalten:

- Der Timer wird gestoppt.
- Die Anzeige springt auf den Anfangswert zurück.
- Der Fortschrittsbalken wird zurückgesetzt.
- Die Eingabefelder werden wieder aktiviert.
- Der Start-Button zeigt wieder **Start**.

20.4 Testfall 4: Ungültige Eingabe

Ausgangssituation: Der Benutzer gibt ungültige Werte ein, zum Beispiel Buchstaben oder negative Zahlen.

Erwartetes Verhalten:

- Das Programm erkennt die ungültige Eingabe.
- Es wird keine Countdown-Zeit gestartet.
- Eine Fehlermeldung wie **Ungültig** wird angezeigt.

20.5 Testfall 5: Sekunden größer als 59

Ausgangssituation: Der Benutzer gibt bei Sekunden einen Wert größer als 59 ein.

Erwartetes Verhalten:

- Die Eingabe wird als ungültig erkannt.
- Der Countdown startet nicht.
- Die Oberfläche bleibt unverändert.

20.6 Testfall 6: MP3-Datei fehlt

Ausgangssituation: Die Datei `alarm.mp3` ist nicht vorhanden oder der Pfad ist falsch.

Erwartetes Verhalten:

- Der Countdown läuft trotzdem bis zum Ende.
- Beim Abspielen der MP3-Datei tritt eine Ausnahme auf.
- Die Fehlermeldung wird in der Konsole ausgegeben.

Stolperfallen

Typischer Fehler:

Wenn nur der Normalfall getestet wird, bleiben Fehler bei Eingaben oder Dateipfaden oft unentdeckt. Deshalb sollten auch ungültige Eingaben und fehlende Dateien geprüft werden.

20.7 Zusammenfassung

Die Testfälle zeigen, dass das Programm in den wichtigsten Situationen korrekt reagieren soll. Besonders wichtig sind dabei die Bedienung des Timers, die Eingabepfung und die MP3-Wiedergabe am Ende.

21 Ausblick

Das Countdown-Projekt ist bereits funktionsfähig und übersichtlich aufgebaut. Dennoch lässt es sich an mehreren Stellen sinnvoll erweitern, um die Bedienung komfortabler zu machen oder zusätzliche Funktionen anzubieten.

Merkkasten

Warum ein Ausblick?

Ein Ausblick zeigt, wie sich das Projekt weiterentwickeln lässt. Dadurch wird deutlich, dass die aktuelle Version eine gute Grundlage für weitere Funktionen bildet.

21.1 Mögliche Erweiterungen

Das Projekt könnte zum Beispiel um folgende Funktionen ergänzt werden:

- **Light Mode:** zusätzlich zum Dark Mode ein helles Design
- **Sound-Auswahl:** verschiedene MP3-Dateien für das Countdown-Ende
- **Slider-Eingabe:** Minuten und Sekunden per Schieberegler einstellen
- **Speichern letzter Einstellungen:** zuletzt verwendete Zeit merken
- **Eigener Alarm-Dialog:** ein Popup-Fenster beim Ablauf des Countdowns
- **Mehrsprachigkeit:** Oberfläche auf Deutsch und Englisch

21.2 Technische Erweiterungen

Auch technisch gibt es weitere Möglichkeiten:

- Trennung von Oberfläche und Logik in mehrere Klassen
- Verwendung von **JavaFX** statt **Swing**
- Einbindung von Icons und Animationen
- Speicherung der Einstellungen in einer Datei

21.3 Einordnung im Projekt

Der Ausblick eignet sich gut als letzter Abschnitt vor der Zusammenfassung. Er zeigt, dass das Projekt nicht nur als Übung gedacht ist, sondern auch als Grundlage für weitere Verbesserungen dienen kann.

Stolperfallen

Typischer Fehler:

Ein Ausblick sollte keine neuen, bereits umgesetzten Funktionen wiederholen. Er soll nur mögliche zukünftige Erweiterungen nennen.

22 Zusammenfassung

Das Countdown-Projekt zeigt, wie sich mit Swing, WindowBuilder und einer zusätzlichen MP3-Bibliothek eine moderne Desktop-Anwendung umsetzen lässt. Die Oberfläche ist übersichtlich aufgebaut und verbindet eine klare Zeitanzeige mit Eingabefeldern, Fortschrittsbalken und runden Buttons.

Merkkasten

Wichtige Erkenntnisse

Das Projekt verbindet mehrere zentrale Java-Themen: grafische Oberflächen, Ereignisverarbeitung, eigene Klassen, Timer-Logik und die Einbindung externer Bibliotheken.

22.1 Was im Projekt umgesetzt wurde

Im Verlauf des Projekts wurden folgende Bausteine erstellt:

- Installation und Nutzung von WindowBuilder
- Einbindung von JLayer für MP3-Sound
- Aufbau einer klaren Projektstruktur in Eclipse
- Gestaltung einer Dark-Mode-Oberfläche
- Umsetzung von runden Buttons mit eigener Klasse
- Countdown-Logik mit Start, Pause, Fortsetzen und Reset
- Fortschrittsbalken und Zeitdarstellung im Format **MM:SS**

22.2 Mögliche Erweiterungen

Das Projekt kann später noch erweitert werden, zum Beispiel um:

- einen eigenen Sound-Auswahldialog
- weitere Designvarianten wie Light Mode
- eine Eingabe per Slider
- das Speichern letzter Einstellungen

22.3 Abschluss

Der Countdown ist damit ein gutes Beispiel dafür, wie sich Java, Swing und WindowBuilder in einem praxisnahen Projekt kombinieren lassen. Die Anwendung ist einfach zu bedienen und gleichzeitig technisch gut erweiterbar.

A Anhang: Quellcode und Erläuterungen

Im Anhang finden Sie die vollständigen Codeblöcke des Countdown-Projekts sowie kurze Erklärungen zu den wichtigsten Klassen und Methoden. Der Anhang ergänzt den Hauptteil der Arbeit und dient dazu, die technische Umsetzung nachvollziehbar darzustellen.

Merkkasten

Zweck des Anhangs Der Anhang enthält den vollständigen Quellcode und die dazugehörigen Erläuterungen, damit die Umsetzung des Projekts auch im Detail nachvollzogen werden kann.

A.1 Inhalt des Anhangs

Der Anhang ist in die folgenden Teile gegliedert:

- Quellcode der Hauptklasse
- Quellcode der Klasse `RoundButton`
- Erläuterung der wichtigsten Methoden
- Hinweise zu verwendeten Bibliotheken und Dateien

A.2 Hinweis zur Lesbarkeit

Die Codeblöcke sind bewusst vollständig angegeben, damit Sie das Projekt direkt nachvollziehen oder in einer eigenen Umgebung nachbauen können. Die Erklärungen fassen die wichtigsten Funktionen jeweils kurz zusammen.

B Quellcode der Hauptklasse

In diesem Anhang wird der vollständige Quellcode der Hauptklasse noch einmal dargestellt und kurz erläutert. Die Klasse `CountdownWindowBuilder` enthält die grafische Oberfläche, die Countdown-Logik, die Steuerung über die Buttons sowie die MP3-Wiedergabe am Ende des Countdowns.

B.1 Vollständiger Quellcode

```
1  import javax.swing.plaf.Player;
2
3  import javax.swing.*;
4  import javax.swing.border.EmptyBorder;
5  import java.awt.*;
6  import java.io.FileInputStream;
7
8  public class CountdownWindowBuilder extends JFrame {
9
10     private JPanel contentPane;
11     private JLabel titleLabel;
12     private JLabel timeLabel;
13     private JTextField minutesField;
14     private JTextField secondsField;
15     private RoundButton startPauseButton;
16     private RoundButton resetButton;
17     private JProgressBar progressBar;
18
19     private Timer timer;
```

```

20     private int initialSeconds = 0;
21     private int remainingSeconds = 0;
22     private boolean running = false;
23
24     private final Color bg = new Color(24, 24, 28);
25     private final Color panelBg = new Color(32, 32, 38);
26     private final Color text = new Color(235, 235, 235);
27     private final Color accent = new Color(70, 130, 255);
28     private final Color green = new Color(46, 125, 50);
29     private final Color red = new Color(198, 40, 40);
30     private final Color fieldBg = new Color(45, 45, 52);
31
32     public static void main(String[] args) {
33         SwingUtilities.invokeLater(() -> {
34             try {
35                 CountdownWindowBuilder frame = new CountdownWindowBuilder();
36                 frame.setVisible(true);
37             } catch (Exception e) {
38                 e.printStackTrace();
39             }
40         });
41     }
42
43     public CountdownWindowBuilder() {
44         setTitle("Countdown");
45         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46         setBounds(100, 100, 560, 420);
47         setLocationRelativeTo(null);
48         setResizable(false);
49
50         contentPane = new JPanel();
51         contentPane.setBorder(new EmptyBorder(20, 20, 20, 20));
52         contentPane.setBackground(bg);
53         setContentPane(contentPane);
54         contentPane.setLayout(new BorderLayout(15, 15));
55
56         JPanel centerPanel = new JPanel(new GridLayout(3, 1, 12, 12));
57         centerPanel.setOpaque(false);
58
59         titleLabel = new JLabel("Countdown", SwingConstants.CENTER);
60         titleLabel.setFont(new Font("SansSerif", Font.BOLD, 30));
61         titleLabel.setForeground(text);
62
63         timeLabel = new JLabel("00:00", SwingConstants.CENTER);
64         timeLabel.setFont(new Font("SansSerif", Font.BOLD, 58));
65         timeLabel.setForeground(accent);
66
67         progressBar = new JProgressBar(0, 100);
68         progressBar.setStringPainted(true);
69         progressBar.setValue(100);
70         progressBar.setString("100%");
71         progressBar.setForeground(accent);
72         progressBar.setBackground(panelBg);
73         progressBar.setBorderPainted(false);
74         progressBar.setPreferredSize(new Dimension(0, 26));
75
76         centerPanel.add(titleLabel);
77         centerPanel.add(timeLabel);
78         centerPanel.add(progressBar);
79

```

```

80     JPanel inputPanel = new JPanel(new FlowLayout(FlowLayout.CENTER,
81         12, 0));
82     inputPanel.setOpaque(false);
83
84     JLabel minLabel = new JLabel("Minuten:");
85     minLabel.setForeground(text);
86     minLabel.setFont(new Font("SansSerif", Font.PLAIN, 15));
87
88     JLabel secLabel = new JLabel("Sekunden:");
89     secLabel.setForeground(text);
90     secLabel.setFont(new Font("SansSerif", Font.PLAIN, 15));
91
92     minutesField = new JTextField("0", 4);
93     secondsField = new JTextField("10", 4);
94     styleField(minutesField);
95     styleField(secondsField);
96
97     inputPanel.add(minLabel);
98     inputPanel.add(minutesField);
99     inputPanel.add(secLabel);
100    inputPanel.add(secondsField);
101
102    JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER,
103        18, 0));
104    buttonPanel.setOpaque(false);
105
106    startPauseButton = new RoundButton("Start");
107    resetButton = new RoundButton("Reset");
108    styleRoundButton(startPauseButton, green);
109    styleRoundButton(resetButton, red);
110    resetButton.setEnabled(false);
111
112    buttonPanel.add(startPauseButton);
113    buttonPanel.add(resetButton);
114
115    JPanel bottomPanel = new JPanel(new GridLayout(2, 1, 10, 10));
116    bottomPanel.setOpaque(false);
117    bottomPanel.add(inputPanel);
118    bottomPanel.add(buttonPanel);
119
120    contentPane.add(centerPanel, BorderLayout.CENTER);
121    contentPane.add(bottomPanel, BorderLayout.SOUTH);
122
123    timer = new Timer(1000, e -> {
124        remainingSeconds--;
125        updateDisplay();
126
127        if (remainingSeconds <= 0) {
128            timer.stop();
129            playMp3("alarm.mp3");
130            timeLabel.setText("Fertig!");
131            progressBar.setValue(0);
132            progressBar.setString("0%");
133            running = false;
134            startPauseButton.setText("Start");
135        }
136    });
137
138    startPauseButton.addActionListener(e -> {
139        if (!running) {
140            if (remainingSeconds <= 0) {

```

```

139         try {
140             int minutes = Integer.parseInt(minutesField.getText().trim());
141             int seconds = Integer.parseInt(secondsField.getText().trim());
142
143             if (minutes < 0 || seconds < 0 || seconds > 59) {
144                 timeLabel.setText("Ungültig");
145                 return;
146             }
147
148             initialSeconds = minutes * 60 + seconds;
149             if (initialSeconds <= 0) {
150                 timeLabel.setText("Zeit > 0");
151                 return;
152             }
153
154             remainingSeconds = initialSeconds;
155         } catch (NumberFormatException ex) {
156             timeLabel.setText("Ungültig");
157             return;
158         }
159     }
160
161     updateDisplay();
162     timer.start();
163     running = true;
164     startPauseButton.setText("Pause");
165     resetButton.setEnabled(true);
166     minutesField.setEnabled(false);
167     secondsField.setEnabled(false);
168 } else {
169     timer.stop();
170     running = false;
171     startPauseButton.setText("Fortsetzen");
172 }
173 });
174
175 resetButton.addActionListener(e -> {
176     timer.stop();
177     running = false;
178     remainingSeconds = initialSeconds;
179     updateDisplay();
180     timeLabel.setText("00:00");
181     progressBar.setValue(100);
182     progressBar.setString("100%");
183     minutesField.setEnabled(true);
184     secondsField.setEnabled(true);
185     startPauseButton.setText("Start");
186     resetButton.setEnabled(false);
187 });
188 }
189
190 private void updateDisplay() {
191     int minutes = remainingSeconds / 60;
192     int seconds = remainingSeconds % 60;
193     timeLabel.setText(String.format("%02d:%02d", minutes, seconds));
194
195     if (initialSeconds > 0) {
196         int percent = (int) ((remainingSeconds * 100.0) / initialSeconds);

```

```

197     progressBar.setValue(percent);
198     progressBar.setString(percent + "%");
199 }
200 }
201
202 private void playMp3(String filePath) {
203     new Thread(() -> {
204         try (FileInputStream fis = new FileInputStream(filePath)) {
205             Player player = new Player(fis);
206             player.play();
207         } catch (Exception e) {
208             e.printStackTrace();
209         }
210     }).start();
211 }
212
213 private void styleField(JTextField field) {
214     field.setFont(new Font("SansSerif", Font.PLAIN, 16));
215     field.setHorizontalAlignment(JTextField.CENTER);
216     field.setPreferredSize(new Dimension(60, 32));
217     field.setBackground(fieldBg);
218     field.setForeground(text);
219     field.setCaretColor(text);
220     field.setBorder(BorderFactory.createLineBorder(new Color(80, 80,
221         90), 1));
222 }
223
224 private void styleRoundButton(RoundButton button, Color color) {
225     button.setBackground(color);
226     button.setForeground(Color.WHITE);
227     button.setFont(new Font("SansSerif", Font.BOLD, 14));
228     button.setPreferredSize(new Dimension(90, 90));
229 }

```

B.2 Erklärung der wichtigsten Teile

- **Variablen:** Speichern Fenster, Eingabefelder, Buttons, Timer und Zustände.
- **Konstruktor:** Baut das Fenster und die Oberfläche auf.
- **Timer:** Reduziert jede Sekunde die verbleibende Zeit.
- **updateDisplay():** Aktualisiert die Anzeige im Format MM:SS.
- **playMp3():** Spielt die MP3-Datei in einem separaten Thread ab.
- **ActionListener:** Steuert Start, Pause, Fortsetzen und Reset.

B.3 Hinweis

Damit dieser Code funktioniert, müssen die Klasse `RoundButton`, die Datei `alarm.mp3` und die `JLayer`-Bibliothek im Projekt vorhanden sein.

C RoundButton-Klasse

Die Klasse `RoundButton` sorgt dafür, dass die Buttons im Countdown-Projekt rund dargestellt werden. Sie erweitert `JButton` und überschreibt die Darstellung des Buttons.

C.1 Vollständiger Quellcode

```
1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.MouseAdapter;
4  import java.awt.event.MouseEvent;
5  import java.awt.geom.Ellipse2D;
6
7  public class RoundButton extends JButton {
8      private Shape shape;
9
10     public RoundButton(String text) {
11         super(text);
12         setContentAreaFilled(false);
13         setFocusPainted(false);
14         setBorderPainted(false);
15         setOpaque(false);
16         setForeground(Color.WHITE);
17         setFont(new Font("SansSerif", Font.BOLD, 14));
18
19         addMouseListener(new MouseAdapter() {
20             @Override
21             public void mouseEntered(MouseEvent e) {
22                 setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
23             }
24         });
25     }
26
27     @Override
28     protected void paintComponent(Graphics g) {
29         Graphics2D g2 = (Graphics2D) g.create();
30         g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
31             RenderingHints.VALUE_ANTIALIAS_ON);
32
33         if (getModel().isArmed()) {
34             g2.setColor(getBackground().darker());
35         } else if (getModel().isRollover()) {
36             g2.setColor(getBackground().brighter());
37         } else {
38             g2.setColor(getBackground());
39         }
40
41         g2.fillOval(0, 0, getWidth(), getHeight());
42         super.paintComponent(g2);
43         g2.dispose();
44     }
45
46     @Override
47     public boolean contains(int x, int y) {
48         if (shape == null || !shape.getBounds().equals(getBounds())) {
49             shape = new Ellipse2D.Float(0, 0, getWidth(), getHeight());
50         }
51         return shape.contains(x, y);
52     }
53 }
```

C.2 Erklärung der wichtigsten Teile

- **extends JButton:** Die Klasse übernimmt das Verhalten eines normalen Buttons.

- **paintComponent()**: Zeichnet den Button als runde Fläche.
- **contains()**: Sorgt dafür, dass nur der runde Bereich klickbar ist.
- **MouseListener**: Ändert den Mauszeiger beim Überfahren des Buttons.

C.3 Hinweis

Die Klasse sollte im gleichen Projekt wie die Hauptklasse liegen, damit sie direkt verwendet werden kann.

D Wichtige Methoden im Überblick

In diesem Abschnitt werden die wichtigsten Methoden der Hauptklasse kurz erläutert. Sie übernehmen zentrale Aufgaben wie die Aktualisierung der Anzeige, das Abspielen der MP3-Datei und die Steuerung des Countdowns.

Merkkasten

Warum eine Methodenübersicht?

Eine kurze Erklärung der wichtigsten Methoden hilft dabei, den Quellcode schneller zu verstehen. Besonders bei einer größeren Klasse ist es sinnvoll, die Aufgaben der einzelnen Methoden getrennt zu beschreiben.

D.1 Methode updateDisplay()

Die Methode `updateDisplay()` aktualisiert die Zeitanzeige im Format **MM:SS**. Sie berechnet aus der verbleibenden Gesamtzeit die Minuten und Sekunden und schreibt den formatierten Wert in das Zeit-Label.

```

1  private void updateDisplay() {
2      int minutes = remainingSeconds / 60;
3      int seconds = remainingSeconds % 60;
4      timeLabel.setText(String.format("%02d:%02d", minutes, seconds));
5
6      if (initialSeconds > 0) {
7          int percent = (int) ((remainingSeconds * 100.0) / initialSeconds);
8          progressBar.setValue(percent);
9          progressBar.setString(percent + "%");
10     }
11 }

```

D.2 Methode playMp3(String filePath)

Die Methode `playMp3()` spielt die MP3-Datei am Ende des Countdowns ab. Dazu wird die Datei mit einem `FileInputStream` geöffnet und über die JLayer-Klasse `Player` wiedergegeben. Die Wiedergabe läuft in einem separaten Thread, damit die grafische Oberfläche nicht blockiert wird.

```

1  private void playMp3(String filePath) {
2      new Thread(() -> {
3          try (FileInputStream fis = new FileInputStream(filePath)) {
4              Player player = new Player(fis);
5              player.play();
6          } catch (Exception e) {
7              e.printStackTrace();
8          }
9      }).start();

```

D.3 Methode `styleField(JTextField field)`

Diese Methode formatiert die Eingabefelder für Minuten und Sekunden. Sie legt Schriftart, Ausrichtung, Hintergrundfarbe, Textfarbe und Rahmen fest, damit beide Felder optisch einheitlich aussehen.

D.4 Methode `styleRoundButton(RoundButton button, Color color)`

Mit dieser Methode werden die runden Buttons formatiert. Sie setzt die Hintergrundfarbe, die Schriftfarbe, die Schriftart und die Größe des Buttons. Dadurch wirken die Schaltflächen einheitlich und passen zum Dark-Mode-Design.

D.5 Timer-Logik im Konstruktor

Der Timer wird im Konstruktor mit einer Verzögerung von einer Sekunde erstellt. Bei jedem Durchlauf wird die verbleibende Zeit um eins reduziert und anschließend die Anzeige aktualisiert. Sobald die Zeit bei null angekommen ist, wird der Timer gestoppt und der MP3-Sound abgespielt.

Stolperfallen

Typischer Fehler:

Wenn die Methoden zwar im Code vorhanden sind, aber nicht aufgerufen werden, ändert sich die Anzeige nicht oder der Sound wird nicht abgespielt. Prüfen Sie deshalb immer, ob die Methoden an der richtigen Stelle verwendet werden.